

Recursion

Definition

Function calling itself.

A problem is broke into smaller problem(subproblem) and the solution is generated using the subproblems.

Example

Here's an example of how recursion can be used to calculate the sum of the first **N** natural numbers:

```
sum(N) = 1 + 2 + 3 + 4 + ..... + N-1 + N
```

```
sum(N) = sum(N-1) + N
```

Here, **sum(N-1)** is smaller instance of same problem that is **sum(N)**.

sum(N-1) is known as a subproblem.

How to write a recursive code?

We can solve any recursive problem using below magic steps:

1. **Assumptions** : Decide what the function does for the given problem.
2. **Main logic** : Break the problem down into smaller subproblems to solve the assumption.
3. **Base case** : Identify the inputs for which we need to stop the recursion.

1. Problem Statement

Given a positive integer N, find the factorial of N.

Quiz

If N = 5, find the factorial of N.

100

120

150

125

Solution :

Assumptions: Create a function which -

- Takes an integer value N as parameter.
- Calculates and returns N!
- return type should be integer.

Main logic:

- The factorial of N is equal to N times the factorial of (N-1)
- We made assumption that sum(N) calculates and return factorial of N natural number. Similarly, sum(N-1) shall calculate and return the factorial of N-1 natural number.

```
factorial(N) = N * factorial(N-1)
```

Base case: The base case is when N equals 0, i.e., **0! = 1**

For example,

When we perform, factorial(N) = N * factorial(N-1), value of N keeps on decreasing by 1.

```
N -> (N-1) -> (N-2) ..... 2 -> 1
```

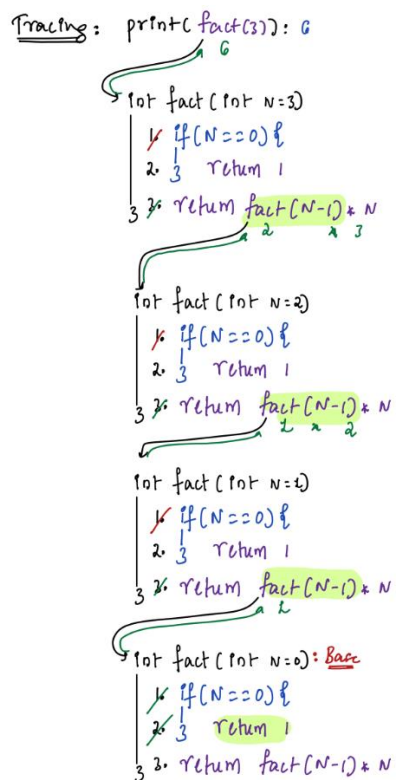
for N = 1 as well as 0, the factorial is 1.

So, we can write base case for $N = 0$ which will also cover for $N = 1$.

Pseudocode

```
int factorial(int N) {  
    // base case  
    if (N == 0) {  
        return 1;  
    }  
    // recursive case  
    return N * factorial(N-1);  
}
```

Dry Run



Time Complexity

No. of function calls = $N+1$

Each function call time is = $O(1)$

$\text{Fact}(5) \rightarrow \text{fact}(4) \rightarrow \text{fact}(3) \rightarrow \text{fact}(2) \rightarrow \text{fact}(1) \rightarrow \text{fact}(0)$

TC = $O(N)$

Space Complexity

SC = $O(N)$

2. Problem Statement

Given N number, print numbers from 1 to N in increasing order.

Example: Inc (5)

Output: 1 2 3 4 5

Pseudocode

```
void inc (N)
if (N==1){
    print(1)
    return
}
inc (N-1)
print(N)
}
```

Dry run:

Time Complexity

$O(\text{Total \# of function calls} * \text{TC per function calls})$

TC = $O(N)$

Space Complexity

$O(\text{max depth of recursion tree/stack} + \text{SC per function call})$

SC = $O(N)$

3.Problem Statement

Given N number, print numbers from N to 1 in decreasing order.

Example: Inc (5)

Output: 5 4 3 2 1

Pseudocode

```
void dec (N)
if (N==1){
    print(1)
    return
}
print(N)
dec (N-1)
}
```

Time Complexity

$O(\text{Total \# of function calls} * \text{TC per function calls})$

TC = $O(N)$

Space Complexity

$O(\text{max depth of recursion tree/stack} + \text{SC per function call})$

SC = $O(N)$

4.Problem Statement

Given a positive integer N, write a function to compute the Nth number in the Fibonacci sequence using recursion.

The Fibonacci sequence is a series of numbers in which each number is the sum of the two preceding numbers.

The sequence goes like this:

N	0	1	2	3	4	5	...
Fibonacci(N)	0	1	1	2	3	5	...

Example

Input = 6
Output = 8

Quiz

If $N = 7$, find the N th number in the fibonacci sequence.

8
5
13
10

Assumptions: Create a function which -

- Takes an integer value N as parameter.
- calculates and returns N th number in the fibonacci sequence.
- return type should be integer.

Main logic: Recursively compute the $(N-1)$ th and $(N-2)$ th numbers in the sequence and add them together to get the n th number.

```
fibonacci(N) = fibonacci(N - 1) + fibonacci(N - 2)
```

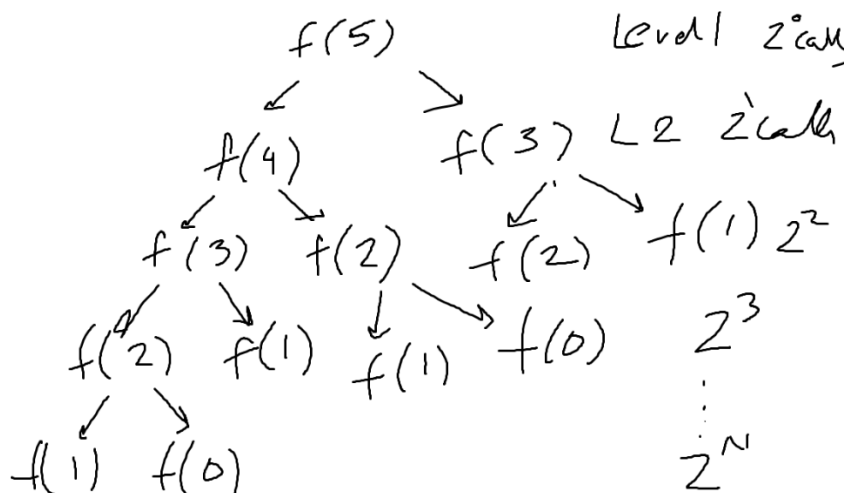
Base case: If N is 0 or 1, return the corresponding value in the Fibonacci sequence.

- Since, according to the definition of fibonacci sequence, the smallest two possible values of N are $N = 1$ & $N = 2$, therefore, stop recursion as soon as N becomes 0 or 1.

Pseudocode

```
int fibonacci(int N) {  
    // base case  
    if (N == 0 || N == 1) {  
        return N;  
    }  
    // recursive case  
    return fibonacci(N - 1) + fibonacci(N - 2);  
}
```

Time Complexity



There is roughly 2^X calls at each level

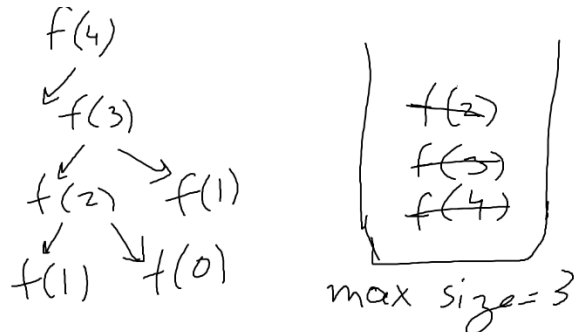
Total levels = N

Total function calls = $2^0 + 2^1 + 2^2 + 2^3 + \dots + 2^{N-1}$

This is Geometric Progression = $(2^0(2^N - 1)) / 2 - 1 = 2^N - 1$

TC = $O(2^N)$

Space Complexity

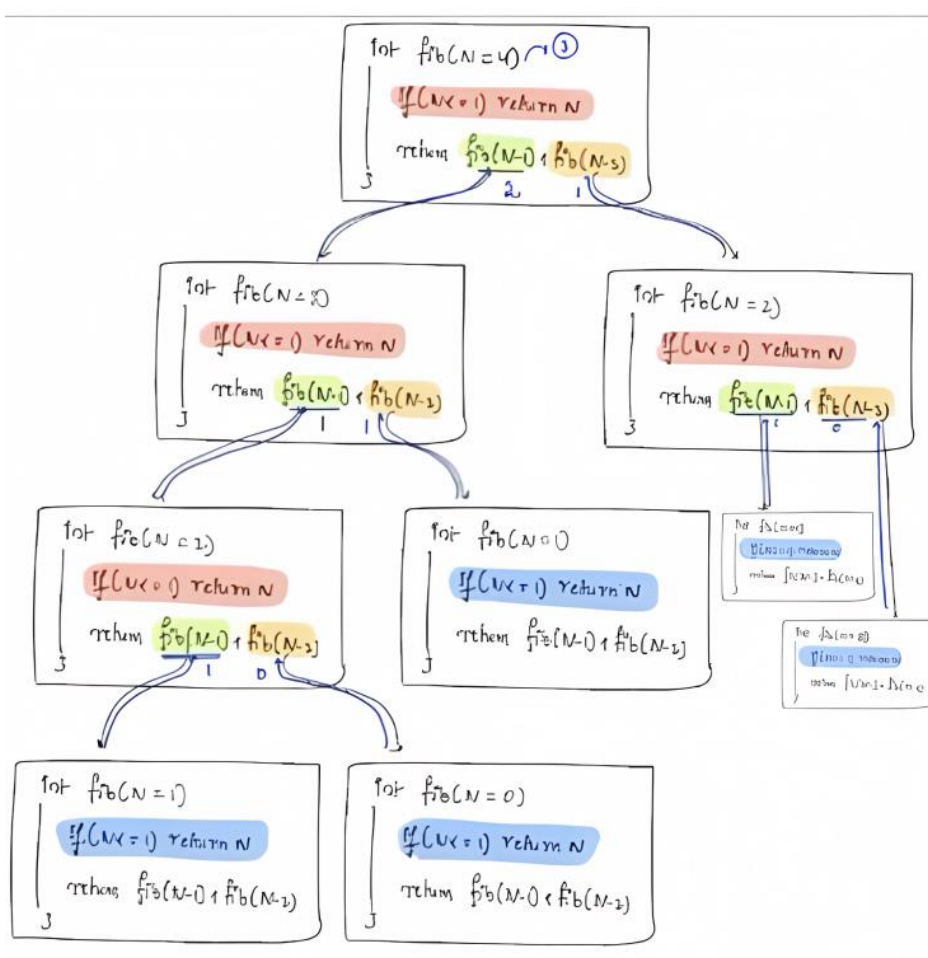


Fib of 4 will call fib(3) and this will be pushed in stack. When fib(2) is returned, it will be removed from stack.

At a time, we have max $n-1$ stack size.

SC = $O(N)$

Function Call Tracing



Problem Statement 1– Check Palindrome using Recursion

Write a recursive function that checks whether String A is a palindrome or not. Return 1 if the String A is palindrome, else return 0.

Example Input -1:

A = "naman"

Output = 1

Example Input -2:

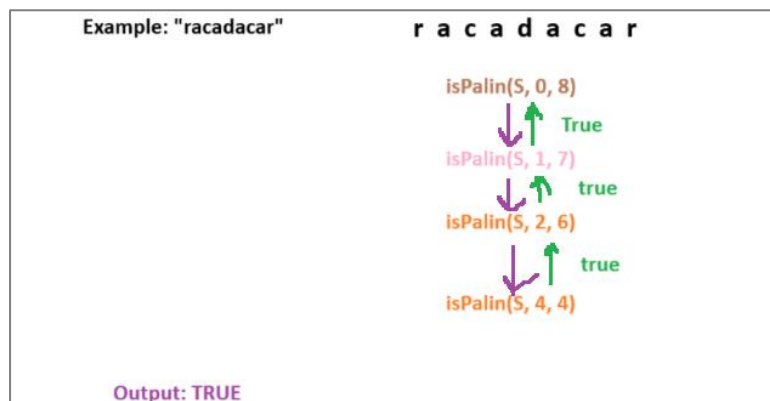
A = "strings"

Output = 0

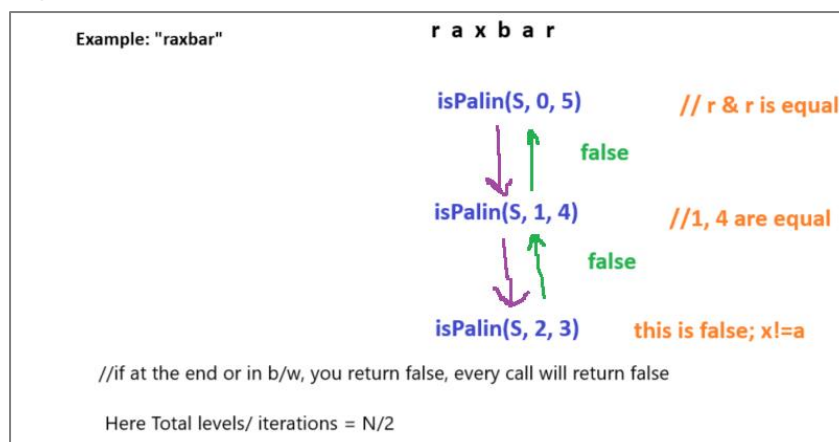
Actual Code

```
public class Solution{
    public int solve(String A){
        int i=0;
        int j = A.length -1;
        return isPalin(A, i, j);
    }
    private int isPalin(String A, int i, int j){
        if( i >= j) return 1;
        if(A.charAt(i) != A.charAt(j)) return 0;
        return isPalin(A, i+1, j-1);
    }
}
```

Dry Run 1:



Dry Run2:



Problem Statement 2– All Indices of Array

Given an array of integers A with N elements and a target integer B, the task is to find all the indices at which B occurs in the array.

Output Format: Return the sorted array of indices.

1.Example Input: A = [1, 2, 3, 4, 5]

B = 1

Output: 0

2.Example Input: A = [8, 9, 5, 6, 5, 5]

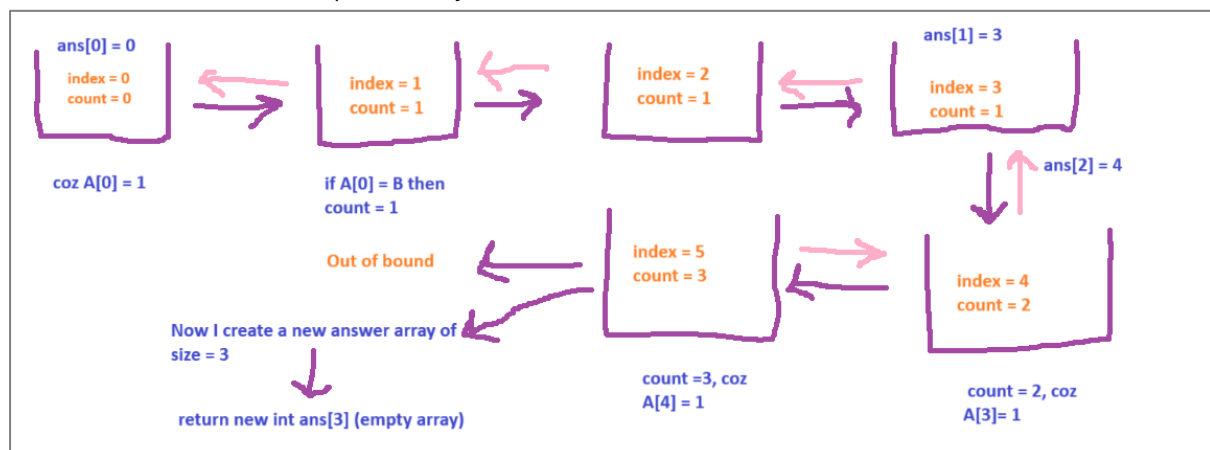
B = 5

Output: [2, 4, 5]

Actual Code

```
public class Solution{
    public int [] allIndices(int [] A, int B){
        int index = 0, count = 0;
        return find(A, B, index, count);
    }
    private int [] find(int [] A, int B, int index, int count){
        if(index == A.length) return new int [count];
        int [] ans; // initializing array
        if(A[index] == B){
            ans = find(A,B, index+1, count + 1); //ans will contain indices of occurrence of B in the array
            ans[count] = index;
        } else {
            ans = find(A, B, index+1, count);
        }
        return ans;
    }
}
```

Note: if we do count++, this will permanently increase the count.



Problem Statement 3– Sum of Digits

Given a number A, we find the sum of its digits using recursion.

Example Input: A = 46

Output: 10

Actual Code

```
public class Solution{
    public int solve(int A){
        if(A==0) return 0;
        return (A%10) + solve(A/10);
    }
}
```

Problem Statement 4– Decreasing Increasing in one Function

Print N numbers in decreasing Order and then in Increasing order. You are required to print the numbers from N to 1.

Note: Print a new line after printing the output.

Example Input: 4

Output: 4 3 2 1 1 2 3 4

Actual Code

```
public class Solution{
    public void DecThenInc(int A){
        inc(A);
        System.out.println();
    }
    private void inc(int A){
        if(A == 0) return;
        System.out.print(A + " ");
        inc(A-1);
        System.out.print(A + " ");
    }
}
```

Problem Statement 5- Tower of Hanoi

There are n disks placed on tower A of different sizes.

Goal

Move all disks from tower A to C using tower B if needed.

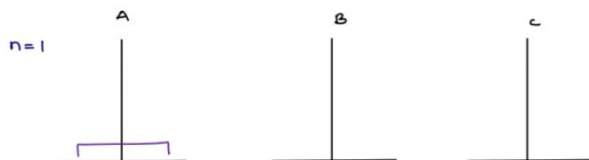
Constraint

- Only 1 disk can be moved at a time.
- Larger disk can not be placed on a small disk at any step.

Print the movement of disks from A to C in minimum steps.

Example 1

Input: N = 1



Explanation:

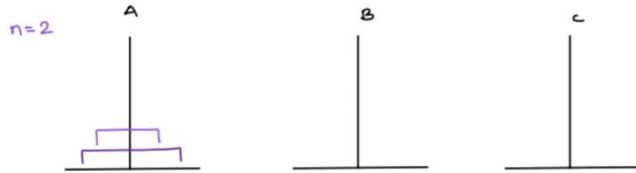


Output:

1: A -> C

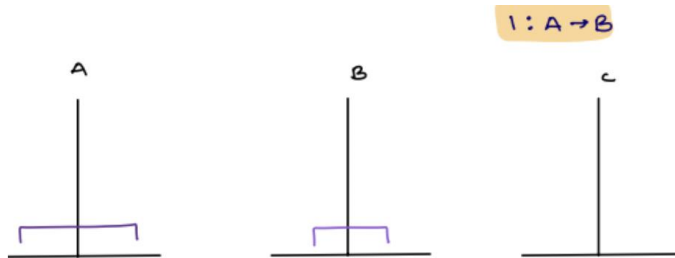
Example 2

Input: N = 2

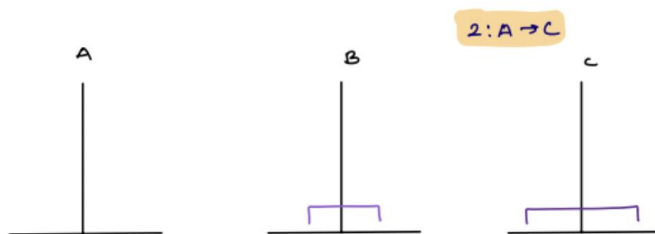


Explanation:

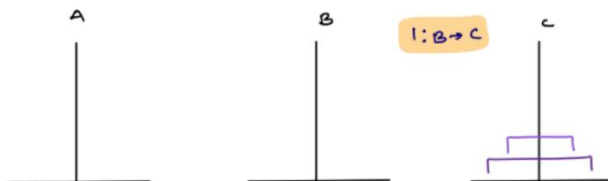
1: $A \rightarrow B$



2: $A \rightarrow C$



1: $B \rightarrow C$



Output:

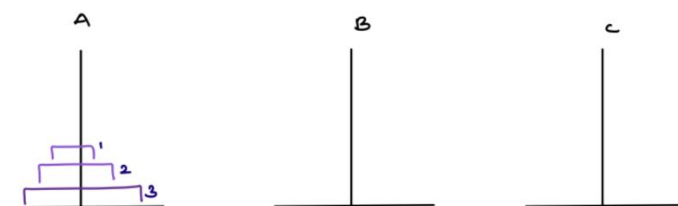
1: $A \rightarrow B$

2: $A \rightarrow C$

1: $B \rightarrow C$

Example 3

Input: $N = 3$

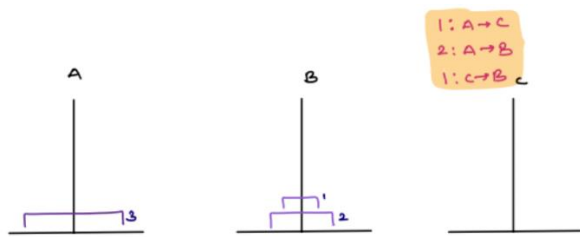


Explanation:

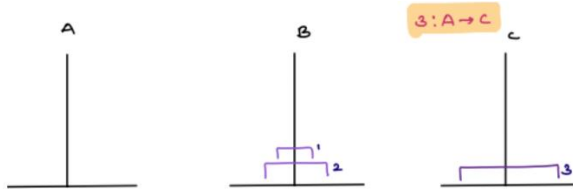
1: A → C

2: A → B

1: C → B



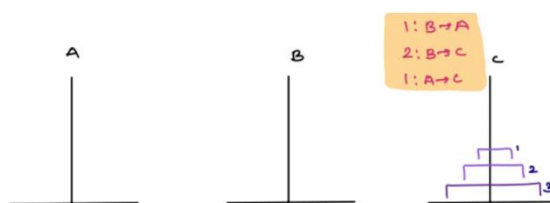
3: A → C



1: B → A

2: B → C

1: A → C



Output:

1: A → C

2: A → B

1: C → B

3: A → C

1: B → A

2: B → C

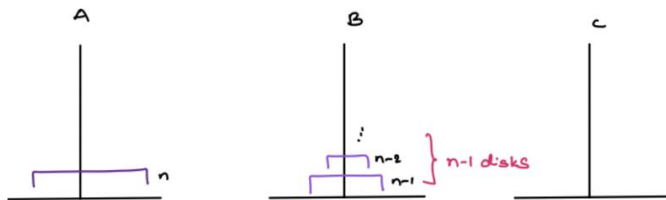
1: A → C

n disks



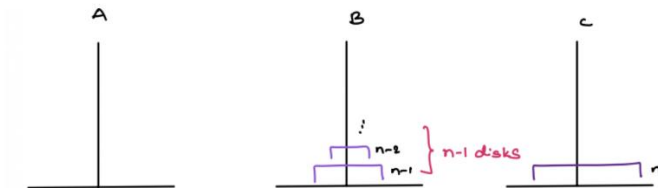
Step 1:

Move (n-1) disks from A to B



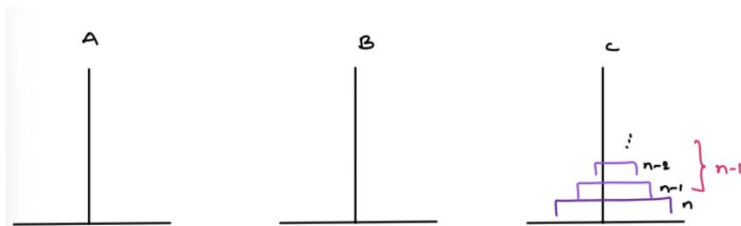
Step 2:

Move n disk to C



Step 3:

Move (n-1) disks from B to C



Pseudocode

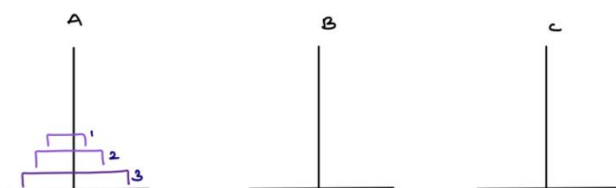
```

src temp dest
void TOH(int n, A, B, C){
  1. if(n==0)
    return;
  2. TOH(n-1, A, C, B); // move n-1 disks A->B
  3. print(n:A->C); // moving n disk A->C
  4. TOH(n-1, B, A, C); // move n-1 disks B->C
}

```

Dry Run

n = 3



TOH(3) A B C X X X X

2: TOH(2, a, c, b) : 1 x x x

2: TOH(ⁿ1, ^{sre}a, ^τb, ^{dest}c) : 1 x x x

2: TOH(0, a, c b) : return

3: print(1: a → c)

4: TOH(0, b, a, c) : return

print(2: a → b)

4: TOH(ⁿ1, ^sc, ^ra, ^Db) : x x x x

2: TOH(0, c, b, a) : return

print(1: c → b)

4: TOH(0, a, c, b) : return

3: print(3: A → C)

4: TOH(ⁿ2, ⁶B, ^τA, ^DC) : x 2 3 4

Output:

1: $A \rightarrow C$
 2: $A \rightarrow B$
 1: $C \rightarrow B$
 3: $A \rightarrow C$
 1: $B \rightarrow A$
 2: $B \rightarrow C$
 1: $A \rightarrow C$

RECURSION -2

5.Problem Statement

Given 2 integers a & n, find a^n using recursion.

Example Input: $a = 2, n = 3$

$$2^3 = 2 \cdot 2 \cdot 2 = 8$$

Pseudocode

```
int pow(a, n){
    if(n==0) return 1;
    return pow(a, n-1) * a;
}
```

$\text{pow}(5, 3)$ $25 \times 5 = 125$
 $\downarrow \uparrow \text{return } 25$ $\text{TC} = O(N)$
 $\text{pow}(5, 2)$ $5 \times 5 = 25$ $\text{SC} = O(N)$
 $\downarrow \uparrow \text{return } 5$
 $\text{pow}(5, 1)$ $1 \times 5 = 5$
 $\downarrow \uparrow \text{return } 1$
 $\text{pow}(5, 0)$

Alternative approach

1. $a^n = a^{n/2} * a^{n/2}$ // if n is even
2. $a^n = a^{n/2} * a^{n/2} * a$ // if n is odd

Optimize approach (Fast Exponentiation)

```

int pow(a,n){
    if(n == 0) return 1;
    x = pow(a, n/2);
    if(n%2 == 0){
        return x * x;
    } else{
        return x * x * a;
    }
}

```

}

$\text{pow}(2, 5)$ $\text{return } 4 \times 4 \times 2 = 32$
 $\downarrow \uparrow 4$
 $\text{pow}(2, 2)$ $\text{return } 2 \times 2 = 4$
 $\downarrow \uparrow 2$
 $\text{pow}(2, 1)$ $n=1$ $\text{return } 1 \times 1 \times 2 = 2$
 $\downarrow \uparrow \text{return } 1$
 $\text{pow}(2, 0)$

Time Complexity

TC = $O(\log N)$

SC = $O(\log N)$

6.Problem Statement

Given an array of integers. Write a recursive function to print all elements.

Example:- $A = [1, 2, 3, 4, 5]$

Output: 1 2 3 4 5

Actual Code

```

public class Solution{
    public void printArray(int [] A){
        int index = 0;
        solve(A, index);
        System.out.println();
    }
}

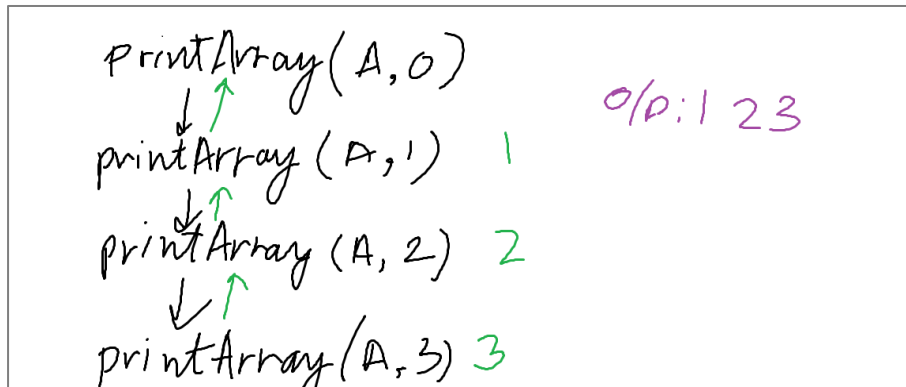
```

```

private void solve(int[] A, int index){
    if(index == A.length){
        return;
    }
    System.out.print(A[index]+ " ");
    solve(A, index +1);
}
}

```

Dry Run:



TC = O(N)

SC = O(N)

7.Problem Statement – Fast Power

Given two +ve integers A & B. Implement Fast Power function to compute A^B .

Example: Input: A= 2, B = 10

Output: 1024

Explanation: $2^{10} = 2^5 * 2^5$
 $= 32 * 32 = 1024.$

```

public class Solution{
    public long power(int A, int B){
        return pow(A, B);
    }
    private long pow(int A, int B){
        if(B == 0) return 1;
        long x = pow(A, B/2);
        if(B%2 == 0){
            return x * x;
        } else{
            return x * x * A;
        }
    }
}

```

8.Problem Statement – Is Magic?

Given a number **A**, check if it is a **magic number** or not.

A number is said to be a magic number if the sum of its digits is calculated till a single digit recursively by adding the sum of the digits after every addition. If the single digit comes out to be 1, then the number is a magic number.

Output Format

Return an 1 if the given number is magic else return 0.

Example Input: A = 83557

Output: 1

Example Explanation:

Sum of digits of (83557) = 28

Sum of digits of (28) = 10

Sum of digits of (10) = 1.

Single digit is 1, so it's a magic number. Return 1.

```
public class Solution {
    public int solve(int A) {
        int result = magic(A);
        return (result == 1)? 1:0;
    }
    private int magic(int A){
        if(A < 10) return A;
        int B = 0;
        while (A > 0){
            B += A%10;
            A /= 10;
        }
        return magic(B);
    }
}
```

9.Problem Statement – Max of an Array Using Recursion?

Given an array A of size N, write a recursive function that returns the maximum element of the array.

Output Format: A single integer is the maximum value of the array.

Example Input:

A = [12, 10, 3, 4, 5]

Output:12

```
public class Solution {
    public int getMax(int[] A) {
        int index = 0;
        int B = Integer.MIN_VALUE;
        return maxrec(A, index, B);
    }
    private int maxrec(int []A, int index, int B){
        if(index == A.length) return B;
        B = Math.max(B, A[index]);
        return maxrec(A, index+1, B);
    }
}
```

10.Problem Statement - First Index using Recursion

You are given an array A of size N. Write a recursive function that returns the first index at which an integer B is found in the array.

NOTE: If B is not found anywhere in the array then return -1.

Output Format: Return the Index of B from the list in integer format if B is present in the array, else return -1.

Example Input: A = [-3, 5, 6, 2]

B = 6

Output: 2

Explanation:

At index 0 we have -3

At index 1 we have 5

At index 2 we have 6 which is equal to B

```
public class Solution {
    public int FirstIndex(int[] A, int B) {
        int index = 0, idx = -1;
        return firstidx(A, B, index, idx);
    }
    private int firstidx(int []A, int B, int index, int idx){
        if(index == A.length) return idx;
        if (B == A[index]){
            idx = index;
            return idx;
        }
        return firstidx(A, B, index+1, idx);
    }
}
```

11.Problem Statement - Last Index using Recursion

You are given an array A of size N. Write a recursive function that returns the last index at which an integer B is found in the array.

NOTE: If B is not found anywhere in the array then return -1.

Output Format: Return the last Index of B from the list in integer format if B is present in the array, else return -1

Example Input 1 : A = [6, 5, 6, 2]

B = 6

Output: 2

Example Explanation:

At index **0** we have **6** which is equal to **B**, but it's not the last occurrence.

At index **1** we have **5**

At index **2** we have **6** which is equal to B, and it's the last occurrence as well.

So the output is **2** and not **0**.

Actual Code

```
public class Solution {
    public int LastIndex(int[] A, int B) {
        int index = 0, idx = -1;
        return lastidx(A, B, index, idx);
    }
    private int lastidx(int []A, int B, int index, int idx){
        if (index == A.length) return idx;
        if (B == A[index]) idx = index;
        return lastidx(A, B, index+1, idx);
    }
}
```


Question

find sum of first N natural no. using recursion.

$$\text{sum}(N) = 1 + 2 + 3 + \dots + N-1 + N$$

$$\text{sum}(5) = \boxed{1 + 2 + 3 + 4} + 5$$

$\text{sum}(4)$

$$\text{sum}(5) = \text{sum}(4) + 5$$

$$\boxed{\text{sum}(N) = \text{sum}(N-1) + N}$$

$\hookrightarrow \text{sum}(N-2) + N-1$
:

Code

```
int sum(N) {  
    //base case  
    if(N==1) return 1  
    return sum(N-1) + N  
}
```

function call tracing

```
int add( int x, int y) {  
    return x+y
```

```
}
```

```
int mul( int x, int y) {  
    return x+y
```

```
}
```

```
int sub( int x, int y) {  
    return x-y
```

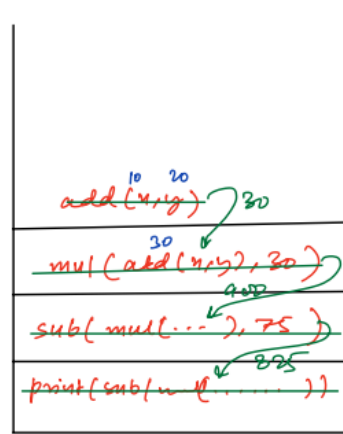
```
}
```

```
main() {
```

```
    x=10, y=20
```

```
    print( sub( mul( add(x,y), 30), 75) )
```

```
}
```



Call
Stack

Output: 825

Question

Given a prime integer N , find factorial of N .

$$N=3 \quad \text{fact}(3) = 1 \times 2 \times 3 = 6$$

$$N=5 \quad \text{fact}(5) = 1 \times 2 \times 3 \times 4 \times 5 = 120$$

Step 1

$$\text{fact}(N) = 1 \times 2 \times 3 \times \dots \times N$$

Step 2

$$\text{fact}(N) = \text{fact}(N-1) \times N$$

Step 3

$$\text{fact}(0) = 1$$

Code

```
int fact(N) {  
    if (N == 0) return 1  
    return fact(N-1) * N  
}
```



Question

Given a time number N , find N^{th} fibonacci no.

N	0	1	2	3	4	5	6	7
$\text{fib}(N)$	0	1	1	2	3	5	8	13
						$\uparrow$	$\uparrow$	
						$\text{fib}(5)$	$\text{fib}(6)$	

$$\text{fib}(7) = \text{fib}(6) + \text{fib}(5)$$

1. $\text{fib}(N) \rightarrow N^{\text{th}}$ fibonacci no.

$$2. \text{fib}(N) = \text{fib}(N-1) + \text{fib}(N-2)$$

$$3. \text{fib}(0) = 0$$

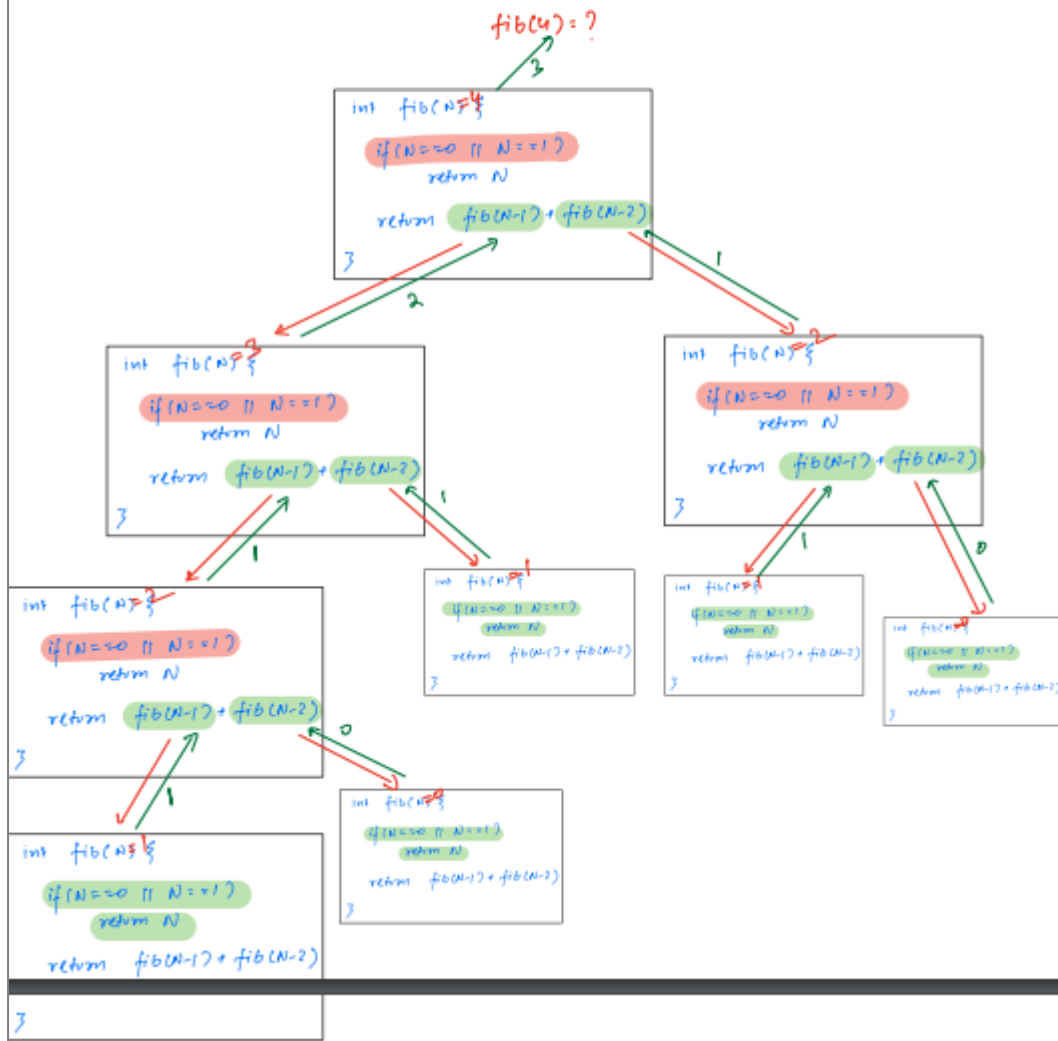
$$\text{fib}(1) = 1$$

$$\text{fib}(1) = \text{fib}(0) + \text{fib}(-1)$$

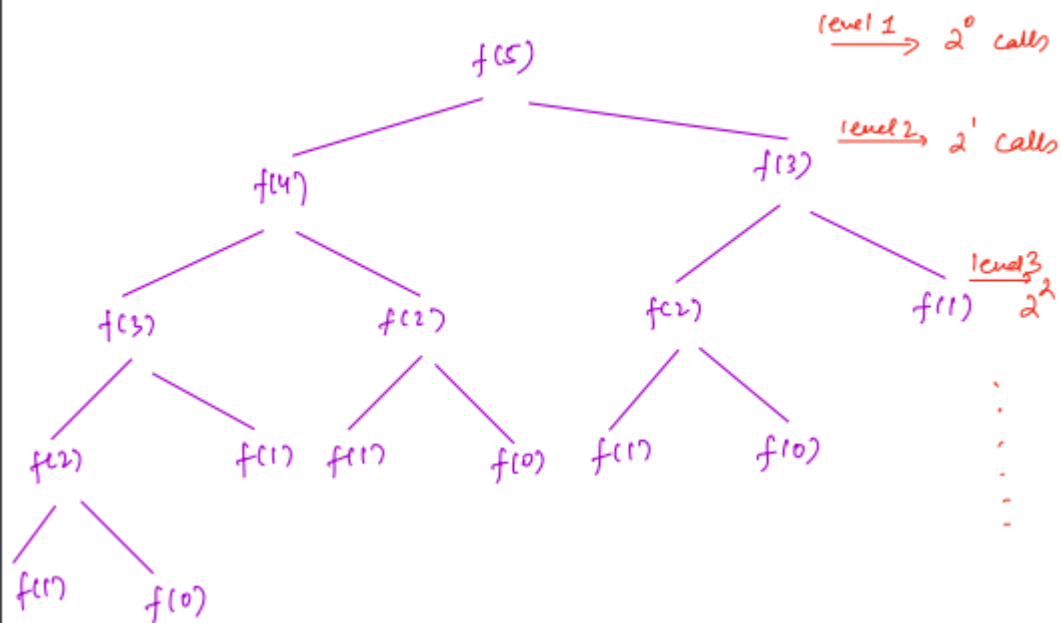
$$\text{fib}(2) = \text{fib}(1) + \text{fib}(0)$$

Code

```
int fib(N) {  
    if (N == 0 || N == 1)  
        return N  
    return fib(N-1) + fib(N-2)  
}
```



Time Complexity



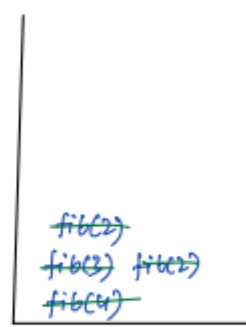
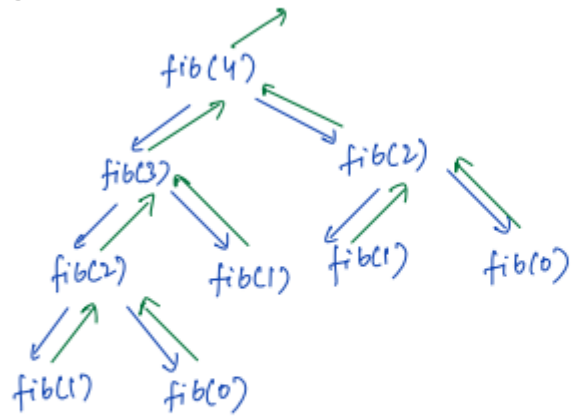
there is $\sim 2^x$ calls at each level

total levels = N

$$\begin{aligned} \text{total } f^m \text{ calls} &= 2^0 + 2^1 + 2^2 + \dots + 2^{N-1} \\ &= 2^0 \left(\frac{2^N - 1}{2 - 1} \right) \quad \left[\text{sum } a \left(\frac{r^n - 1}{r - 1} \right) \right] \\ &= 2^N - 1 \end{aligned}$$

$$\boxed{TC = O(2^N)}$$

Space Complexity



max size = 3

$$SC = O(N)$$